

Обзор проекта: университетская дата-платформа

Создается интегрированная платформа для управления разнородными университетскими данными.

Жукова Арина Александровна, студенческий билет 1132239120, НПИбд-01-23

Обзор проекта

Цель: Создание современной дата-платформы для управления данными университета по принципам Data Mesh

Задачи:

Объединить разрозненные источники данных (LMS, CSV, события кампуса)

Обеспечить потоковую обработку событий в реальном времени

Построить многослойное Lakehouse-хранилище

Внедрить семантический слой и embedded-аналитику

Автоматизировать развёртывание и тестирование через CI/CD

Ключевые принципы:

Инфраструктура как код (Terraform)

Отказоустойчивые ETL/ELT-пайплайны с валидацией

Data Mesh: 3 домена (academic_performance, campus_infrastructure, student_engagement)



Контекст и предпосылки

Проблемы

Разрозненные данные: LMS API, CSV-выгрузки, события кампуса

Отсутствие единой аналитической платформы

Нет мониторинга загрузки кампуса в реальном времени

Ручная подготовка данных для ML-моделей

Решение

Единая платформа на базе Yandex Cloud

Data Lake (S3) + Lakehouse (Bronze/Silver/Gold)

Потоковая обработка через Managed Kafka + ClickHouse

Семантический слой Cube.js + дашборд Streamlit

Проектирование: доменно-ориентированная архитектура и инфраструктура как код (IaC)

01

Обособлены три домена:
academic_performance —
успеваемость студентов
campus_infrastructure —
загрузка аудиторий и корпусов
student_engagement —
вовлечённость (события
кампуса)
Каждый домен отвечает за
отдельный набор данных и
бизнес-логики.

02

Data Product для успеваемости
строится на LMS API и CSV-
выгрузках; создаются
уникальные и полные витрины
признаков student_features для
ML-моделей.
Метрики качества: полнота
student_id = 100%,
уникальность ключей

03

Инфраструктура
развёртывается с помощью
Terraform: выделены S3 бакеты
по слоям, виртуальная машина
под Airflow, развернут Managed
Kafka кластер, настроены
отдельные DAG для обработки.

Подтверждение работы ключевой инфраструктуры и модульность развёртывания



Работа облачных компонентов подтверждена запуском VM airflow-vm

В Yandex Cloud создана виртуальная машина с IP 51.250.82.7 для запуска Orchestrator Airflow, обеспечивающего управление ETL-процессами. Это критический узел платформы, доказавший стабильность работы.

DAGsCluster ActivityDatasetsSecurity▼Browse▼Admin▼Docs▼

23:05 UTC AU

Search DAGs

☒ Auto-refresh

	DAG	Owner	Runs	Schedule	Last Run	Next
☒	etl_to_raw_layer etlrawvalidation	airflow	4	@daily	2026-05-07, 20:15:23	2026-
☐	lakehouse_transformations featuresgoldlakehouse	airflow	4	@daily	2026-05-07, 20:50:57	2026-
☐	stream_events real-timestream	airflow		*15 ****		2026-

«

<

1

>

»

Showing 1-3 of 3 DAGs

S3 бакеты организованы по уровням Lakehouse

```
arich@HONOR:~/uni-data-platform$ echo "=== S3 Бакеты ==="
aws s3 ls --endpoint-url=https://storage.yandexcloud.net

echo ""
echo "=== Raw-слой ==="
aws s3 ls s3://uni-raw-data-b1gualomaijt1a9no9sa/ --recursive --endpoint-url=https://storage.yandexcloud.net | head -10

echo ""
echo "=== Gold-слой ==="
aws s3 ls s3://uni-gold-layer-b1gualomaijt1a9no9sa/ --recursive --endpoint-url=https://storage.yandexcloud.net
=== S3 Бакеты ===
2026-05-07 20:22:10 uni-raw-data-b1gualomaijt1a9no9sa
2026-05-07 20:22:10 uni-bronze-layer-b1gualomaijt1a9no9sa
2026-05-07 20:22:10 uni-silver-layer-b1gualomaijt1a9no9sa
2026-05-07 20:22:10 uni-gold-layer-b1gualomaijt1a9no9sa

=== Raw-слой ===
2026-05-07 23:15:31      3656 api/courses/2026-05-07/courses.parquet
2026-05-07 23:15:36      4664 api/grades/2026-05-07/grades.parquet
2026-05-07 23:15:27      3873 api/students/2026-05-07/students.parquet
2026-05-07 23:15:39      4777 csv/rooms/2026-05-07/rooms.parquet
2026-05-07 23:15:39      4221 csv/schedule/2026-05-07/schedule.parquet
2026-05-07 23:15:40      3038 csv/teachers/2026-05-07/teachers.parquet
2026-05-07 22:48:31 409804800 docker-images/airflow273.tar
2026-05-07 22:41:54 163780608 docker-images/postgres15.tar
2026-05-07 23:26:21 400395283 docker-images/spark.tgz

=== Gold-слой ===
2026-05-07 23:54:54      3124 aggregations/people_count/2026-05-07_20-54-53.parquet
2026-05-07 23:46:34      5310 course_performance/course_performance.parquet
2026-05-07 23:54:53      8431 events/2026-05-07_20-54-53/events.parquet
2026-05-07 23:46:34      6440 student_features/student_features.parquet
arich@HONOR:~/uni-data-platform$
```

Бакеты uni-raw-data-*, uni-bronze-layer-*, uni-silver-layer-*, uni-gold-layer-* структурируют данные по слоям хранения, обеспечивая надежность и разделение данных по этапам обработки.

Работа стримингового конвейера и ClickHouse подтверждена

```
=== Managed Kafka Cluster ===  
Имя: uni-kafka-cluster  
Статус: RUNNING  
Окружение: PRODUCTION  
  
=== Топики ===  
uni_events: partitions=3, replication=1
```

Managed Kafka кластер находится в состоянии RUNNING с топиком uni_events

```
=== ClickHouse таблицы ===  
people_count  
student_features  
  
=== People Count ===  
1      50  
2      46  
  
=== Student Features ===  
1      Иванов Иван      74.333336      good  
2      Петрова Анна      67      satisfactory  
3      Сидоров Павел      89.5      excellent  
4      Козлова Мария      71      good  
5      Новиков Дмитрий  65.75      satisfactory
```

ClickHouse (Docker) содержит таблицы uni.people_count и uni.student_features для оперативного и аналитического доступа.

Terraform — структура модулей

📁 .terraform

📄 .terraform.lock

📄 main

📄 outputs

📄 terraform.tfstate

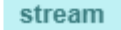
📄 terraform.tfstate.backup

📄 terraform.tfvars

📄 variables

```
=== Ресурсы в main.tf ===
resource "yandex_storage_bucket" "raw" {
resource "yandex_storage_bucket" "bronze" {
resource "yandex_storage_bucket" "silver" {
resource "yandex_storage_bucket" "gold" {
resource "yandex_vpc_network" "uni_network" {
resource "yandex_vpc_subnet" "uni_subnet" {
resource "yandex_iam_service_account" "vm_sa" {
resource "yandex_resourcemanager_folder_iam_member" "vm_sa_storage" {
resource "yandex_compute_instance" "airflow_vm" {
resource "yandex_mdb_kafka_cluster" "uni_kafka" {
resource "yandex_mdb_kafka_user" "producer" {
resource "yandex_mdb_kafka_user" "consumer" {
resource "yandex_mdb_kafka_topic" "uni_events" {
```

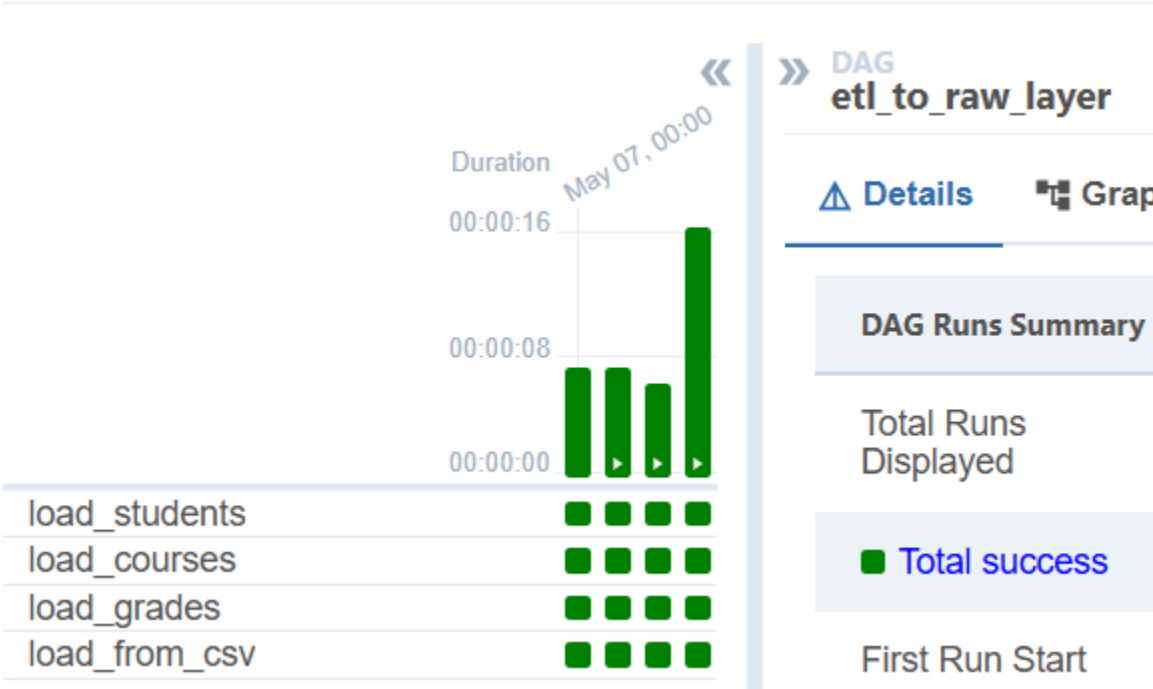
Airflow — отдельные DAG для каждого слоя

 DAG 	Owner 	Runs 
 etl_to_raw_layer   		   
 lakehouse_transformations   		   
 stream_events  		   

ETL/ELT пайплайны: загрузка, retry и валидация качества данных

01 DAG etl_to_raw_layer извлекает данные из LMS API и CSV в S3 Raw слой с до трёх попыток при ошибках, обеспечивая устойчивость к сбоям в потоках данных.

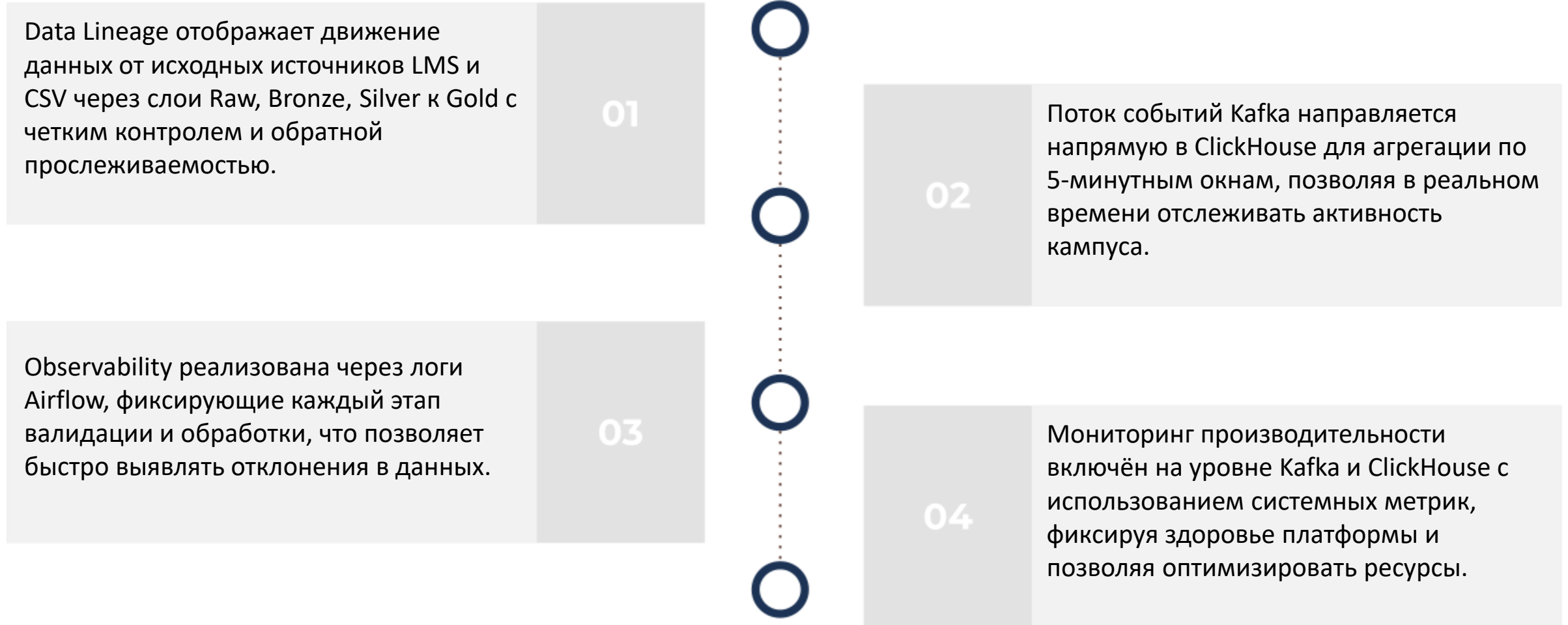
02 Валидация данных исполнена с помощью Great Expectations: проверяется уникальность student_id, корректность оценок и отсутствие NULL в важных полях. Результаты логируются в Airflow, подтверждая качество данных.



```
ubuntu@fhmcoc5rnijsbcb6ofp:~$ docker exec airflow-all grep "
Валидация" /opt/airflow/logs/dag_id=etl_to_raw_layer/run_id=ma
nual__2026-05-07T20:15:23+00:00/task_id=load_students/attempt=1.log
[2026-05-07T20:15:27.691+0000] {logging_mixin.py:154} INFO -
Валидация students пройдена успешно (5 строк)
```

dag_id	run_id	state
etl_to_raw_layer	manual__2026-05-07T20:15:23+00:00	success
etl_to_raw_layer	manual__2026-05-07T20:09:23+00:00	success

Data Observability и Data Lineage: прозрачность преобразований и метрик качества



Архитектурный обзор: ключевые компоненты и их назначение

Основные компоненты платформы обеспечивают весь конвейер данных от хранения до визуализации и аналитики.

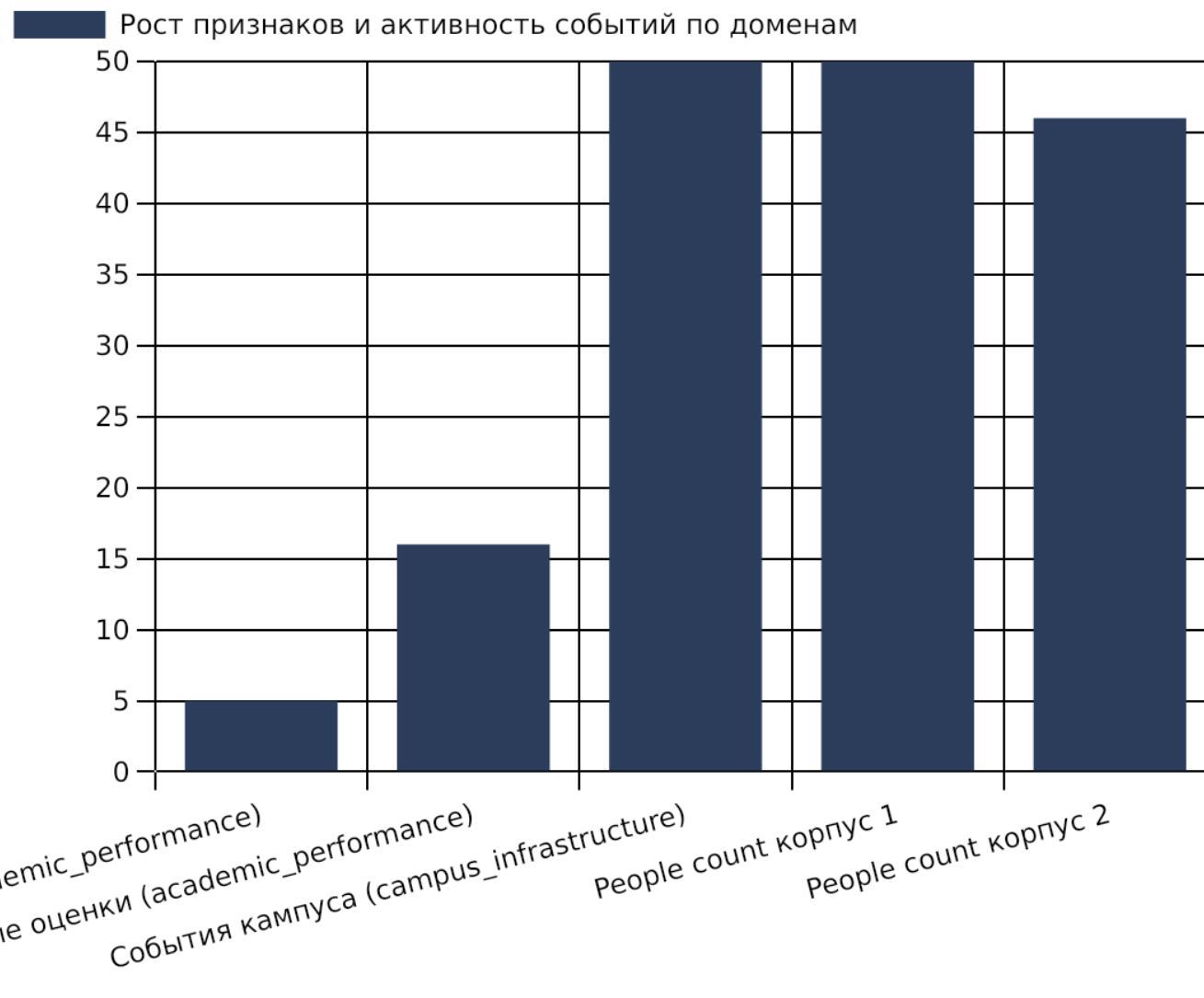
Каждый компонент специализирован для своего этапа обработки, обеспечивая целостность и масштабируемость платформы.

Компонент	Назначение
Yandex S3	Data Lake с четырьмя слоями
Airflow 2.7	Оркестрация ETL/ELT процессов
Managed Kafka	Стриминговая передача событий
ClickHouse 23.3	База для real-time аналитики
Cube.js 1.6	Семантический слой и API
Streamlit	Визуализация и embedded аналитика

Рост объёмов данных и сквозная обработка по доменам

Наблюдается расширение признакового пространства и активное поступление событий в реальном времени, что подкрепляет цель платформы — полное покрытие данных кампуса.

Интеграция batch и стриминговых данных позволяет реализовать сквозную аналитику и поддерживать актуальность данных по доменам.



Lakehouse: построение слоёв Bronze, Silver, Gold и atomic-трансформации

Структурирование слоев и обработка данных

DAG lakehouse_transformations обеспечивает безопасный переход данных через Bronze (сырой), Silver (очищенный) и Gold (агрегированный) слои, реализуя фильтрацию и дедупликацию на практике.

Вычисление агрегатов и показателей

На Gold-слое формируются агрегаты — средний балл, количество курсов, задолженности и уровни успеваемости, что критично для построения ML-признаков и аналитики.

```
=== BRONZE LAYER ===
Загружено: s3://uni-bronze-layer-b1
students/students.parquet - 5 строк
Bronze students: 5 строк
Загружено: s3://uni-bronze-layer-b1
courses/courses.parquet - 5 строк
Bronze courses: 5 строк
Загружено: s3://uni-bronze-layer-b1
grades/grades.parquet - 16 строк
Bronze grades: 16 строк

=== SILVER LAYER ===
Загружено: s3://uni-silver-layer-b1g
students/students.parquet - 5 строк
Silver students: 5 строк
Загружено: s3://uni-silver-layer-b1g
courses/courses.parquet - 5 строк
Silver courses: 5 строк
Загружено: s3://uni-silver-layer-b1g
grades/grades.parquet - 16 строк
Silver grades: 16 строк
```

```
=== GOLD LAYER ===
Students: 5, Grades: 16
Загружено: s3://uni-gold-layer-b1gualomaijt1a9no9sa/st
udent_features/student_features.parquet - 5 строк
Загружено: s3://uni-gold-layer-b1gualomaijt1a9no9sa/co
urse_performance/course_performance.parquet - 5 строк
Features: 5 студентов
Course report: 5 курсов
```

```
=== S3 Gold Layer ===
2026-05-07 23:54:54      3124 aggregations/people_count/2026-05-07_20-54-53.parquet
2026-05-08 02:36:44      5310 course_performance/course_performance.parquet
2026-05-07 23:54:53      8431 events/2026-05-07_20-54-53/events.parquet
2026-05-08 02:36:44      6440 student_features/student_features.parquet
```

```
=== S3 Silver Layer ===
2026-05-08 02:36:44      3228 courses/courses.parquet
2026-05-08 02:36:44      4235 grades/grades.parquet
2026-05-08 02:36:44      3445 students/students.parquet
```

```
=== S3 Bronze Layer ===
2026-05-08 02:36:43      3228 courses/courses.parquet
2026-05-08 02:36:43      4235 grades/grades.parquet
2026-05-08 02:36:43      3445 students/students.parquet
```

Feature Store: автоматизация хранения признаков для ML

student_id	avg_score	courses_taken	debts	performance_level
1	78.5	3	0	good
2	85.0	4	0	excellent
3	55.0	2	2	at_risk

Таблица student_features в Gold-слое содержит ключевые признаки, необходимые для прогнозов и аналитики академической успеваемости.

Автоматизированное создание признаков позволяет повысить качество ML-моделей и обеспечить своевременный доступ к данным.

Потоковая обработка событий: генерация, агрегация и запись в ClickHouse

Архитектура потокового канала

Реализована архитектура потоковой обработки с компонентами: Kafka Producer на Python генерирует события кампуса, которые публикуются в топик uni_events с тремя партициями для устойчивой передачи данных. Данные поступают непрерывно с частотой от 0,5 до 2 секунд, обеспечивая оперативность информации о действиях студентов.

Механизм агрегации и сохранения

Агрегатор обрабатывает входящие события, группируя их по идентификатору корпуса (building_id) в скользящем окне длительностью 5 минут. Каждые 30 секунд результаты агрегации отправляются в таблицу people_count в ClickHouse, что обеспечивает оперативное обновление ключевых метрик и возможность мониторинга в реальном времени.

```
arich@HONOR:~/uni-data-platform$ python3 kafka_producer_full.py &
```

```
[1] 10799
```

```
arich@HONOR:~/uni-data-platform$
```

```
[0] student.submitted.assignment
```

```
[1] student.entered.classroom
```

```
[2] student.submitted.assignment
```

```
[3] student.entered.classroom
```

```
[4] student.entered.classroom
```

```
[5] student.entered.classroom
```

```
arich@HONOR: ~/uni-data-pli
```

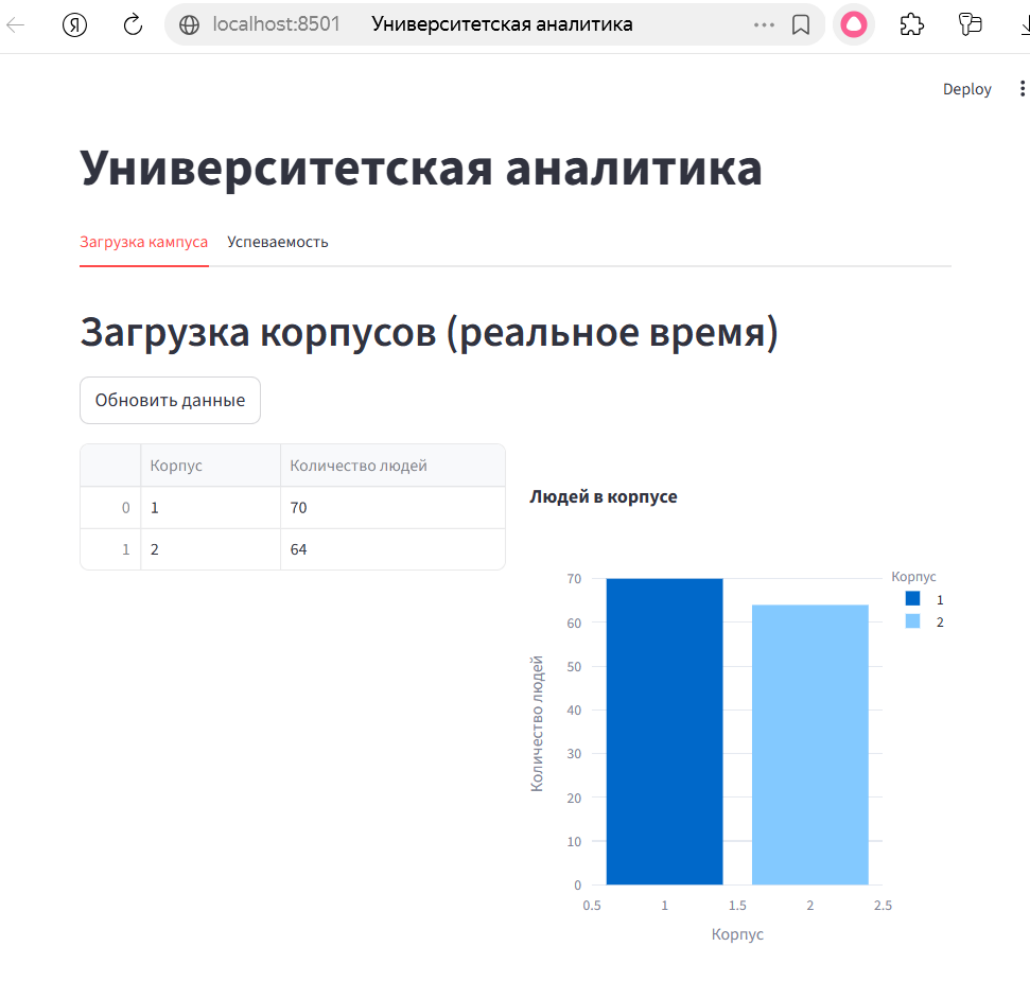
```
arich@HONOR:~/uni-data-platform$ python3
```

```
Агрегатор запущен. Ожидание событий...
```

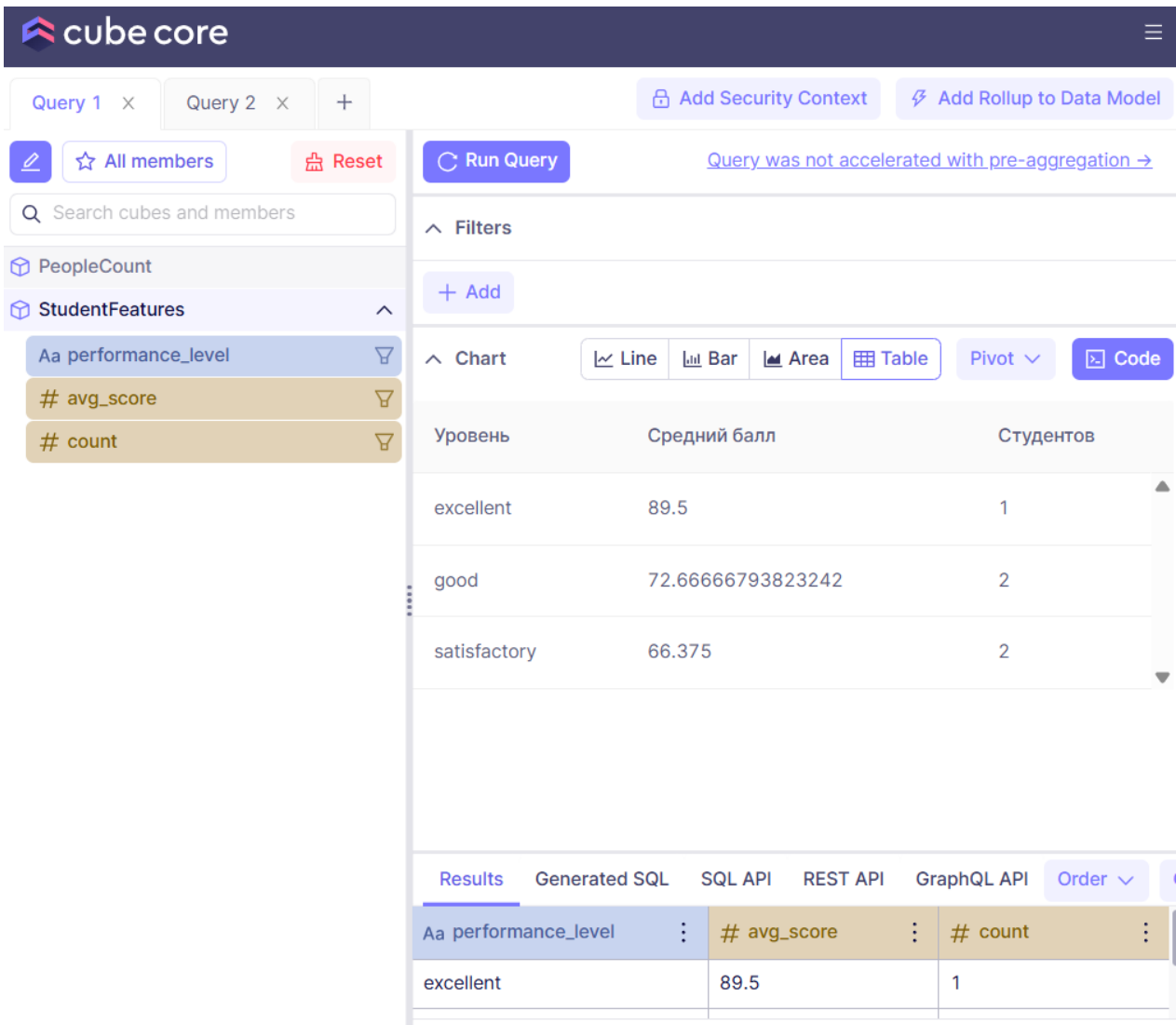
```
Inserted: building=2, count=10
```

```
Inserted: building=1, count=8
```

Визуализация real-time данных и аналитика с помощью Streamlit и Cube.js



Семантический слой и API-интеграция Cube.js для Embedded Analytics



```
=== Cube.js API: PeopleCount ===
{
  "query": {
    "measures": [
      "PeopleCount.total_people"
    ],
    "dimensions": [
      "PeopleCount.building_id"
    ],
    "timeDimensions": [],
    "limit": 10000,
  },
}

=== Cube.js API: StudentFeatures ===
{
  "query": {
    "measures": [
      "StudentFeatures.count",
      "StudentFeatures.avg_score"
    ],
    "dimensions": [
      "StudentFeatures.performance_level"
    ],
    "timeDimensions": [],
    "limit": 10000,
    "timezone": "UTC",
    "filters": [],
    "rowLimit": 10000
  },
}
```

CI/CD для платформы: автоматизация тестирования, сборки и деплоя

01

GitLab CI/CD организован в несколько этапов: тестирование с помощью pytest для проверки корректности data-трансформаций и валидации качества данных через Great Expectations.

02

Процесс сборки включает создание Docker-образов для всех сервисов инфраструктуры, а этап деплоя обеспечивает автоматическое развертывание на тестовом сервере с использованием Terraform.

03

Встроена автоматизация тестирования Airflow DAG, мониторинг исполнения с алартами при сбоях и управление конфигурациями инфраструктуры для обеспечения отказоустойчивости и воспроизводимости релизов.

Документация

ADR.pdf/md

Architecture Decision Records

ADR-001: Yandex Cloud как облачный провайдер

- Статус:** Принято
- Решение:** Использовать Yandex Cloud (Object Storage, Managed Kafka, Compute Cloud)
- Причины:**
- Единая экосистема с API-совместимостью S3
 - Управляемый Kafka без настройки инфраструктуры
 - Российский регион с низкой задержкой

ADR-002: Data Lake на S3 с Parquet

- Статус:** Принято
- Решение:** Хранить данные в S3-совместимом Object Storage в формате Parquet
- Причины:**
- Колоночное хранение = быстрые аналитические запросы
 - Сжатие данных экономит до 80% места
 - Совместимость с Spark/Pandas/ClickHouse

ADR-003: Airflow для оркестрации

data_products.pdf/md

Data Products платформы университета

1. People Count (поточковая аналитика)

- Владелец:** Инфраструктурный отдел
- Источник:** Kafka топик `uni_events`
- Обновление:** Real-time (30-секундные окна)
- SLA:** 99.5% доступности
- Метрики качества:**
- Полнота данных: 100%
 - Задержка доставки: < 5 секунд
 - Точность агрегаций: 100%
- Интерфейсы:**
- Вход: события `student.entered.classroom` из Kafka
 - Выход: таблица `uni.people_count` в ClickHouse
 - API: `Cubejs /cubejs-api/v1/load`

2. Student Features (успеваемость)

- Владелец:** Учебное управление
- Источник:** API LMS, CSV-выгрузки

Data Lineage.pdf/md

Data Lineage

Происхождение данных

LMS API → Raw S3 (students, courses, grades) → Bronze → Silver → Gold (student_features)

CSV → Raw S3 (schedule, rooms, teachers) → Bronze → Silver

Kafka Events → ClickHouse (people_count) — агрегация 5-мин окон

Зависимости между датасетами

Входной датасет	Трансформация	Выходной датасет
Raw students	Bronze	Bronze students
Raw grades	Bronze	Bronze grades
Bronze students	Silver (очистка)	Silver students
Bronze grades	Silver (фильтрация)	Silver grades
Silver students + grades	Gold (агрегация)	Gold student_features
Kafka uni_events	Aggregator	ClickHouse people_count

Сквозная трассировка

README.pdf/md

Университетская дата-платформа

Архитектура

Сырые данные → S3 Raw → S3 Bronze → S3 Silver → S3 Gold → ClickHouse → Cube.js → Streamlit

API LMS / CSV → S3 Raw

Kafka Events → ClickHouse (real-time)

Компоненты

Компонент	Технология	Назначение
Оркестрация	Airflow 2.7	ETL/ELT пайплайны
Хранилище	Yandex S3	Data Lake (4 слоя)
Стриминг	Managed Kafka 3.7	Real-time события
Аналитика	ClickHouse 23.3	Колоночная БД
Семантика	Cube.js 1.6	API для метрик
Дашборд	Streamlit	Embedded Analytics
Инфраструктура	Terraform	IaC
Валидация	Great Expectations	Качество данных

<https://github.com/ZhukovaArina/Development-of-information-and-analytical-systems/tree/master>

Синтез: доказанная работоспособность и перспективы платформы

Реализация всех ключевых компонентов подтверждена успешным функционированием, что создает основу для дальнейшего расширения платформы и интеграции дополнительных доменов, а также повышения автоматизации и надежности системы в целом.

